INT161 Basic Backend Development — สรุปเนื้อหาทั้งหมด (สไลด์ + โค้ดในโปรเจกต์)

> สรุปจากสไลด์ 7 ชุด (Week 1–7) และโค้ดในโฟลเดอร์ `int161/` ทั้งหมด (`my-first-app.js` , `http-server.js` , `basic-rest/` , `test-express/` , `express-template/` , `express-template2/` , `node-express-esm-template/`)
>
> โครงของเอกสาร: แต่ละบทมี ① **แนวคิด** → ② **ต้องทำอะไร (ขั้นตอน)** → ③ **โค้ดแยกส่วน** → ④ **Flow การทำงาน** ท้ายเอกสารมี **"สร้างโปรเจกต์ตั้งแต่ศูนย์ แบบ step-by-step"**, **Checklist สอบ**, และ **บั๊กที่เจอในโค้ดปัจจุบัน**

สารบัญ

- 0. ภาพรวมวิชา
- 1. Week 1 — Introduction: Web Application, HTTP, Node.js
- 2. Week 2 — HTTP Programming Basics (node:http)
- 3. Week 3 — RESTful API & Layered System (Pure Node.js)
- 4. Week 4 — Database Connection (mysql2)
- 5. Week 5 — Express.js Framework
- 6. Week 6 — Error Handling
- 7. Week 7 — Pagination & DTO
- 8. สร้างโปรเจกต์ตั้งแต่ศูนย์ (Step-by-step ครบทุกไฟล์)
- 9. Flow ของ 1 Request ตั้งแต่เข้าจนออก
- 10. Checklist สอบปฏิบัติ (จาก test-runner.mjs)
- 11. บั๊ก/จุดผิดที่เจอในโค้ดปัจจุบัน
- 12. Quick Reference

---

0. ภาพรวมวิชา

INT161 Basic Backend Development (การพัฒนาแบ็กเอนด์พื้นฐาน) — RESTful API Development

เนื้อหาหลัก: Basic RESTful API, การเชื่อมต่อฐานข้อมูล, API Authentication & Security, Sorting, Pagination, Filtering

การประเมินผล

การสอบ	สัดส่วน	รูปแบบ
2nd Exam (Theory)	30%	ปรนัย (เลือก 1 ข้อ) + อธิบาย
2nd Exam (Practice)	20%	เขียนโปรแกรมแก้โจทย์
3rd Exam (Theory)	30%	ปรนัย
3rd Exam (Practice)	20%	เขียนโปรแกรมแก้โจทย์

ตารางเรียน (Group B วันพฤหัส / Group A วันศุกร์)

ครั้ง	หัวข้อ
1	Introduction
2	Node.js vs Bun (HTTP Basics)
3	RESTful API
4	Express framework
5	Connection to Database
—	**1st Examination (7–11 Sep 2026)**
6	Relationship Modeling / ORM
7	Prisma ORM
8	Validation & Error Handling
9	Data Transfer Object (DTO)
10	Filtering, Sorting & Pagination
—	**2nd Examination (19–26 Oct 2026)**
11	JWT: JSON Web Tokens
12	Authentication / Authorization
13–14	Integrated Project Support Topic
—	**3rd Examination (30 Nov – 11 Dec 2026)**

ภาพรวมเส้นทางการเรียน (ทำอะไรก่อน-หลัง)

```
Week1  เข้าใจ Web/HTTP/Node.js → สร้าง server แรกด้วย node:http
  ↓
Week2  อ่าน request (method, url, query, path) → ตอบ response เอง
  ↓
Week3  ออกแบบ REST + แยก layer (router → service → repository) แบบไม่ใช้ framework
  ↓
Week4  เปลี่ยน repository ให้ไปคุยกับ MySQL จริง (mysql2 + pool + prepared statement)
  ↓
Week5  ย้ายจาก node:http มาใช้ Express (router, middleware, req.params/query/body)
  ↓
Week6  ทำ Error handling กลาง (AppError + error-handling middleware)
  ↓
Week7  Pagination middleware + DTO mapper + apiResponse (รูปแบบ JSON มาตรฐาน)
```

---

1. Week 1 — Introduction: Web Application, HTTP, Node.js

1.1 แนวคิด

Web Application = Client ↔ Application Server ↔ Database Server - Client ส่ง **HTTP Request** → Server ประมวลผล → ตอบ **HTTP Response** - Client ไม่จำเป็นต้องเป็น browser เสมอ (mobile app, curl, Postman ก็ได้)

HTTP Request มี 3 ที่เก็บข้อมูล 1. **HTTP Method** (GET / POST / PUT / DELETE / PATCH) 2. **HTTP Headers** (เช่น `Content-Type: application/json`) 3. **HTTP Body** (payload เช่น JSON)

HTTP Response มี 3 ส่วน 1. **Status Line** (เช่น `HTTP/1.1 200 OK`) 2. **Headers** 3. **Message Body (Payload)**

สถาปัตยกรรม Web App

แบบ	ลักษณะ
MPA (Multi-Page App)	Server render HTML ทุกหน้า (Router → Controller → Model → View/Template Engine) รีเฟรชทั้งหน้า
MVC แบบ modern/SPA	Front-end framework เรียก API ด้วย XMLHttpRequest/fetch, Server ส่งแค่ข้อมูล
SPA (Single Page App)	โหลด HTML ครั้งแรกครั้งเดียว จากนั้นคุยกับ Back-end (RESTful API / Microservice) ด้วย **JSON** ผ่าน AJAX — ใช้ backend เดียวกับ Mobile App ได้

JSON (JavaScript Object Notation) — รูปแบบแลกเปลี่ยนข้อมูลที่เบา อ่านง่าย parse ง่าย เป็นอิสระจากภาษา

Web Services 2 แบบ - **SOAP** — Simple Object Access Protocol, ใช้ XML, อธิบายด้วย WSDL - **RESTful** — คิดโดย Roy Fielding (คนเดียวกับที่ออกแบบ HTTP) → **REST เป็น design approach ไม่ใช่ protocol**

Full Stack Developer = เขียนได้ทั้ง browser (JS/React/Vue), server (Java/PHP/Python/Node.js), database (SQL/NoSQL)

1.2 Node.js คืออะไร

- **ไม่ใช่ภาษา** แต่เป็น **runtime** ที่รัน JavaScript ฝั่ง server (แปลง JS → machine code)
- Open source, ฟรี, รันได้หลาย OS
- ทำงานแบบ **asynchronous, single thread** (async/await) → เหมาะกับงาน I/O เยอะ เช่น REST API, chat real-time

ตัวแปลภาษา 3 แบบ (ออกสอบทฤษฎีได้)

แบบ	การทำงาน	ตัวอย่าง	ข้อดี	ข้อเสีย
Compiler	แปลทั้งโปรแกรมเป็น executable ก่อนรัน	C, C++, Go	เร็ว, ซ่อน source code	ต้อง compile ใหม่ต่อ OS/CPU (portable น้อย)
Interpreter	แปลทีละบรรทัดตอนรัน	Python, JS	portable, dev/debug เร็ว	ช้ากว่า, ต้องแจก source code
Hybrid	compile เป็น **byte code** แล้ว VM (Interpreter + JIT) แปลตอนรัน	Java, Kotlin, C#	สมดุลทั้งสองแบบ	—

1.3 ต้องทำอะไร (ขั้นตอน)

1. ติดตั้ง Node.js ผ่าน nvm
2. เขียนไฟล์ `.js` แล้วรันด้วย `node <file>.js`
3. สร้าง Web Server แรก: **import module → create server → อ่าน request / ส่ง response**

```
# ติดตั้ง nvm + node
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.2/install.sh | bash
\. "$HOME/.nvm/nvm.sh"
nvm install 22
node -v    # v22.x
npm -v
```

1.4 โค้ด

(ก) โปรแกรม console — `myfirst.js` (รับ input จาก terminal)

```
const readline = require('node:readline/promises');
const { stdin: input, stdout: output } = require('node:process');

async function askQuestion() {
    const rl = readline.createInterface({ input, output });
    try {
        const name = await rl.question('What is your name? ');
        const birthyear = await rl.question('What is your birth year in B.E.? ');
        const currentYear = new Date().getFullYear();
        const age = currentYear - birthyear + 543;
        console.log(`Thank you ${name}, aged ${age}!`);
    } finally {
        rl.close();   // ปิด stream เสมอ
    }
}
askQuestion();
```

(ข) Web server แรก — `my-first-app.js` (มีในโปรเจกต์)

```
http = require("node:http");
listener = function (request, response) {
  response.writeHead(200, { "Content-Type": "text/html" }); // status + header
  response.end('<h2 style="text-align: center;">Hello World</h2>'); // body แล้วจบ
};
server = http.createServer(listener);
server.listen(3000);
console.log("Server running at http://127.0.0.1:3000/");
```

(ค) อ่านข้อมูลจาก request — `my-second-app.js`

```
listener = function (request, response) {
    response.write('http version: ' + request.httpVersion + '\n');
    response.write('url: ' + request.url + '\n');
    response.write('method: ' + request.method + '\n');
    response.write('headers.user-agent: ' + request.headers["user-agent"] + '\n');
    response.end('headers.host: ' + request.headers.host + '\n');
};
```

(ง) Routing แบบ manual — `my-third-app.js`

```
listener = function (request, response) {
    if (request.url === '/user') {
        response.writeHead(200, {'Content-Type': 'application/json'})
        response.end(JSON.stringify({ name: 'John Doe' }))   // ต้องแปลง object → string
    } else if (request.url === '/') {
        response.writeHead(200)
        response.end('Hello World')
    } else {
        response.writeHead(404)
        response.end('Error: Page Not Found')
    }
};
```

1.5 Flow

```
node my-first-app.js
  → http.createServer(listener)     สร้าง server พร้อมฟังก์ชันจัดการ request
  → server.listen(3000)             เริ่มรอ connection ที่ port 3000
  → Browser เปิด http://localhost:3000
  → listener(request, response) ถูกเรียก
  → response.writeHead(...)         ส่ง status + header
  → response.end(...)               ส่ง body และปิด response
```

---

2. Week 2 — HTTP Programming Basics (node:http)

2.1 แนวคิด

Node.js Architecture (Event Loop) 1. Client ส่ง request (อาจเป็น blocking หรือ non-blocking) 2. Node.js รับ request ใส่ **Event Queue** 3. **Event Loop** หยิบทีละตัว ตรวจว่าเป็นงานง่ายไหม 4. งาน non-blocking (I/O polling) ประมวลผลแล้วตอบกลับ client

Core Built-in Modules ที่ใช้บ่อย: `fs`, `http`, `path`, `os`, `events`, `util`, `stream`, `crypto`, `url`, `querystring`

CJS vs ESM (ออกสอบ)

	CommonJS (CJS)	ES Modules (ESM)
Syntax	`require()` / `module.exports`	`import` / `export`
Loading	synchronous ตอน runtime	parse แบบ static, รองรับ async
Environment	Node.js เท่านั้น	ทั้ง browser และ Node.js
Tree-shaking	ไม่ค่อยรองรับ	รองรับ (ตัดโค้ดที่ไม่ใช้)
ตัวแปรไฟล์	`__dirname`, `__filename`	`import.meta.url`
เปิดใช้	ค่า default	ใส่ `"type": "module"` ใน `package.json` หรือใช้ `.mjs`

→ **โปรเจกต์ใหม่ใช้ ESM**, CJS ใช้เฉพาะ legacy

4 Class หลักของ `node:http`

Class	หน้าที่	method/property สำคัญ
`http.Server`	ตัว server	`listen(port, cb)`, `close(cb)`, event `'request'`
`http.ServerResponse`	ใช้สร้าง response	`writeHead(status, headers)`, `setHeader(name, val)`, `write(chunk)`, `end(data)`
`http.IncomingMessage`	request ที่เข้ามา (เป็น ReadableStream)	`method`, `url`, `headers`
`http.ClientRequest`	ใช้ยิง request ไป server อื่น	`write(chunk)`, `end()`, event `'response'`

HTTP Methods

Method	ใช้ทำ	ตัวอย่าง
GET	ดึงข้อมูล	`/subjects`
POST	สร้างใหม่ (มี body)	`/subjects`
PUT	แก้ไข (มี body)	`/subjects/{id}`
DELETE	ลบ	`/subjects/{id}`

อ่าน Query String ด้วย WHATWG URL API (`URL` และ `URLSearchParams` เป็น global ไม่ต้อง import)

```
http://localhost:3000/user?name=Somchai&fav_subj=Back-end Dev&fav_subj=Front-end Dev
                           └──────────── query string ──────────────────────────────┘
```

2.2 ต้องทำอะไร

1. `import * as http from 'node:http'`
2. เขียน `listener(request, response)`
3. ดึง `method`, `url` จาก request
4. สร้าง `new URL(request.url, \` http://${request.headers.host}`)` เพื่อ parse
5. อ่าน query: `searchParams.get()` (ค่าเดียว) / `getAll()` (หลายค่า)
6. อ่าน path: `pathname.split('/').filter(Boolean)`
7. `writeHead` → `write` (กี่ครั้งก็ได้) → `end` (ครั้งเดียว ต้องเรียกเสมอ)

2.3 โค้ด

(ก) Server พื้นฐาน + event listener

```
import * as http from 'node:http';

const listener = function (request, response) {
    const { url, method } = request;
    response.writeHead(200, { 'Content-Type': 'text/plain' });
    response.end(`You made a ${method} request to ${url}`);   // template literal
};
const server = http.createServer(listener);
server.listen(3000, () => console.log('Server running at http://127.0.0.1:3000/'));

// ผูก event 'request' เพิ่มอีกตัว (ใช้ log)
server.on('request', (req, res) => console.log('Request received:', req.method, req.url));
```

ทดสอบ:

```
curl -i -X PUT http://localhost:3000/user/5 -d '{"name": "John Doe", "role": "Admin"}'
```

(ข) อ่าน Query Parameters

```
const urlObj  = new URL(request.url, `http://${request.headers.host}`);
const params  = urlObj.searchParams;
const name     = params.get('name');          // "Somchai"
const subjects = params.getAll('fav_subj');   // ["Back-end Dev", "Front-end Dev"]

response.write('Name: ' + name + '\n');
subjects.forEach((subject, index) => response.write(`    ${index + 1}, ${subject}\n`));
```

(ค) อ่าน Path Segments (`/users/123` → `["users","123"]`)

```
const pathSegments = urlObj.pathname.split('/').filter(Boolean);
// split('/')      → ["", "users", "123"]
// filter(Boolean) → ตัด "" ทิ้ง เพราะ Boolean("") = false
```

(ง) แบบฝึกหัด: ตารางสูตรคูณ — `http-server.js` (ไฟล์ในโปรเจกต์)

```
import * as http from "node:http";
const listener = function (request, response) {
    response.writeHead(200, { "Content-Type": "text/plain" });
    const { url, method } = request;
    const urlObj = new URL(request.url, `http://${request.headers.host}`);
    const num = urlObj.searchParams.get("number");         // ?number=5
    response.write("Multiplication number for " + num + "\n");
    for (let i = 1; i <= 12; i++) {
        response.write(`    ${num} x ${i} = ${num * i}\n`);
    }
    response.end(`You made a ${method} request to ${url}`);
};
const server = http.createServer(listener);
server.listen(3000, () => console.log("Server running at http://127.0.0.1:3000/"));
```

ทดสอบ: `curl "http://localhost:3000/?number=5"`

2.4 Flow

```
Request: GET /user?name=Somchai
  → listener ถูกเรียก
  → new URL(request.url, host)        parse URL
  → searchParams.get('name')          ได้ "Somchai"
  → response.writeHead(200, ...)      ส่ง status + header (ต้องก่อน write/end)
  → response.write(...) × n           ส่ง body ทีละชิ้น
  → response.end(...)                 ส่งชิ้นสุดท้าย + ปิด
```

---

3. Week 3 — RESTful API & Layered System (Pure Node.js)

3.1 แนวคิด

REST = REpresentational State Transfer (Roy Fielding) - Server ให้เข้าถึง **resource**, Client อ่าน/แก้ resource - แต่ละ resource ระบุด้วย **URI** - Representation นิยม **JSON** (หรือ text, XML)

6 RESTful Principles / Constraints (ออกสอบ)

ข้อ	ความหมาย
Client-Server	แยก client กับ server, server ตอบหรือปฏิเสธ request
Stateless	ทุก request ต้องมีข้อมูลครบในตัว server ไม่จำ context เก่า → client เก็บ session state เอง
Uniform Interface	ใช้ HTTP verbs มาตรฐานกับ resource (POST/GET/PUT/DELETE)
Layered System	ระบบเป็นชั้น ๆ แต่ละชั้นเห็นแค่ชั้นที่ติดกัน
Cacheable	response ต้องระบุว่า cache ได้หรือไม่
Code on demand *(optional)*	server ส่ง code (script) ให้ client รันได้

Uniform Interface — 4 เสาหลัก 1. **Resource Identification** — URI บอก "สิ่งของ" ไม่ใช่ "การกระทำ" (`/api/v1/users`) 2. **Manipulation via Representations** — แลกเปลี่ยน state ผ่าน JSON 3. **Self-Descriptive Messages** — ใช้ HTTP method + header มาตรฐาน (`Content-Type: application/json`) 4. **HATEOAS** — ใส่ hypermedia links บอก client ว่าทำอะไรต่อได้

กฎการตั้ง Endpoint - URI ลงท้ายด้วย **คำนาม (noun)** ไม่ใช่ verb → `/api/users` ✅ `/getUsers` ❌ - ใช้ **พหูพจน์ (plural)** เสมอ - ใช้ HTTP verb บอก action - ส่ง **status code ให้ถูก**

URI	Verb	ความหมาย
`api/products`	GET	ดึงทั้งหมด
`api/products/1`	GET	ดึง id = 1
`api/products/1/orders`	GET	orders ทั้งหมดของ product 1 (sub-collection)
`api/products`	POST	เพิ่มใหม่
`api/products/1`	PUT	แก้ id = 1
`api/products/1`	DELETE	ลบ id = 1

- `/customers` = **collection resource**
- `/customers/{id}` = **singleton resource**
- `/customers/{id}/orders` = **sub-collection resource**

Layered System

```
Front-End  ──{JSON}──►  HTTP/Routing Layer  (Presentation)
                              │
                              ▼
                        Service Layer       (Business Logic)
                              │
                              ▼
                        Data Access Layer   (Repository)  ──►  DB / File
```

ประโยชน์: **Scalability** (เพิ่ม cache/load balancer ได้โดย client ไม่ต้องเปลี่ยน), **Separation of Concerns** (แยก HTTP parsing ออกจาก business logic และ DB), **Security**

3.2 ต้องทำอะไร (โปรเจกต์ `basic-rest/`)

ทำจาก **ล่างขึ้นบน** (repository → service → router → server):

1. `src/repositories/subject-repository.js` — เก็บข้อมูล (array ในหน่วยความจำ) + ฟังก์ชัน findAll / find / removeSubject
2. `src/services/subject-service.js` — เรียก repository (business logic)
3. `src/router.js` — ตรวจ url + method แล้วเรียก service, ตอบ JSON + status code
4. `src/server.js` — สร้าง http server แล้วส่งทุก request ให้ router
5. รัน `node src/server.js`

```
src/
├── server.js       --> Native HTTP Server Setup
├── router.js       --> Route Matching & HTTP Handling
├── services/       --> Business Logic (Framework-Agnostic)
└── repositories/   --> Data Access Layer (File System / DB)
```

3.3 โค้ด

① Repository — `basic-rest/src/repositories/subject-repository.js`

```
const subjects = [
    {id: 'INT100', name: 'IT Fundamentals', credit: 3},
    {id: "INT101", name: "Programming Fundamentals", credit: 3},
    {id: "INT114", name: "Discrete Mathematics", credit: 3},
    // ... (17 วิชา)
];

export function findAll() {
    return subjects;
}

export function find(id) {
    return subjects.find(sj => sj.id === id);
}

export function removeSubject(id) {
    const idx = subjects.findIndex(sj => sj.id === id);
    if (idx < 0) return false;   // ไม่เจอ
    subjects.splice(idx, 1);      // ลบออกจาก array
    return true;
}
```

② Service — `basic-rest/src/services/subject-service.js`

```
import * as repo from '../repositories/subject-repository.js'

export function getAllSubjects()     { return repo.findAll() }
export function getSubjectById(id)   { return repo.find(id) }
export function removeSubjectById(id){ return repo.removeSubject(id) }
```

③ Router — `basic-rest/src/router.js`

```
import * as service from './services/subject-service.js';

export function handleUserRequest(request, response) {
    const {url, method} = request;

    if (url === '/subjects') {                      // collection
        switch (method) {
            case 'GET':
                const subjects = service.getAllSubjects();
                response.writeHead(200, {'Content-Type': 'application/json'});
                response.end(JSON.stringify(subjects));
                break;
            case 'POST':
                response.writeHead(200, {'Content-Type': 'application/json'});
                response.end(JSON.stringify({warning: 'under construction'}));
                break;
        }
    } else if (url.startsWith('/subjects/')) {       // singleton
        const id = url.slice('/subjects/'.length);   // ตัดเอา id
        switch (method) {
            case 'GET':
                const subject = service.getSubjectById(id);
                response.writeHead(200, {'Content-Type': 'application/json'});
                response.end(JSON.stringify(subject));
                break;
            case 'DELETE':
                if (service.removeSubjectById(id)) {
                    response.writeHead(204);          // ลบสำเร็จ: 204 No Content
                    response.end();
                } else {
                    response.writeHead(404, {'Content-Type': 'application/json'});
                    response.end(JSON.stringify({error: 'Subject not found for id = ' + id}));
                }
                break;
        }
    } else {
        response.writeHead(404, {'Content-Type': 'application/json'});
        response.end(JSON.stringify({error: 'Resource not found'}));
    }
}
```

④ Server — `basic-rest/src/server.js`

```
import * as http from 'node:http';
import * as route from './router.js';

const server = http.createServer((req, res) => route.handleUserRequest(req, res));
server.listen(3000, () => console.log('Server running at http://127.0.0.1:3000/'));
server.on('request', (req, res) => console.log('Request received:', req.method, req.url));
```

⑤ ทดสอบ service โดยไม่ผ่าน HTTP — `basic-rest/index.js`

```
import * as service from './src/services/subject-service.js'
service.removeSubjectById("INT101")
console.log(JSON.stringify(service.getAllSubjects(), null, 2));
console.log(JSON.stringify(service.getSubjectById('INT114'), null, 2));
```

> ⚠️ `basic-rest/package.json` ยังไม่มี `"type": "module"` → ถ้ารันแล้วเจอ `Cannot use import statement outside a module` ให้เพิ่มบรรทัดนี้

3.4 Flow

```
DELETE /subjects/INT101
  → server.js         createServer รับ request → route.handleUserRequest
  → router.js         url.startsWith('/subjects/') → id = "INT101", method = DELETE
  → service           removeSubjectById("INT101")
  → repository        findIndex → splice → return true
  → router.js         true → writeHead(204) → end()
  (ถ้า false → 404 + {error})
```

---

4. Week 4 — Database Connection (mysql2)

4.1 แนวคิด

- ใช้ library `mysql2` (asynchronous, non-blocking ผ่าน event loop)
- **Connection** = การเชื่อมต่อ 1 เส้น / **Pool** = ตัวจัดการ connection หลายเส้น (ใช้ซ้ำได้ เร็วกว่า)
- `pool.query(sql, params)` → คืนค่า `[rows, fields]` (array 2 ตัว) → จึงเขียน `const [rows] = await ...`
- ใช้ **Prepared Statement** (`?` placeholder) เพื่อ **ป้องกัน SQL Injection** — ห้ามต่อ string ค่าจาก user เข้าไปใน SQL ตรง ๆ
- ใช้ได้ 2 แบบ: **callback API** หรือ **Promise API (async/await)** ← วิชานี้ใช้แบบนี้ (`mysql2/promise`)

ผลลัพธ์จาก query แต่ละแบบ

SQL	`[result]` ได้อะไร
SELECT	array ของ rows (`rows.length === 0` = ไม่เจอ)
INSERT	`result.insertId` , `result.affectedRows`
UPDATE / DELETE	`result.affectedRows` (0 = ไม่มีแถวถูกแก้/ลบ)

4.2 ต้องทำอะไร (ลำดับ)

1. **สร้าง DB + ตาราง** (ใช้ DB tool ใน WebStorm: สร้าง Data Source → MySQL → user root)
2. `npm install mysql2`
3. สร้างไฟล์ **pool** (`db/pool.js` หรือ `configs/db-config.js`)
4. เขียน **test script** ลอง query ดูก่อน (แล้ว `pool.end()`)
5. แก้ **repository** ให้ query DB จริง (async ทั้งหมด)
6. แก้ **router** ให้มี `getBody()` (อ่าน body จาก stream) และ `writeResponse()`
7. ทำ POST / PUT ให้ใช้งานได้

4.3 โค้ด

① SQL — สร้างตารางและข้อมูล

```
use sampledb;
create table subjects (
    id            int primary key auto_increment,
    subject_code  varchar(8) not null unique,
    subject_title varchar(60),
    credit        int not null
);

insert into subjects (subject_code, subject_title, credit) values
  ('INT 100', 'IT Fundamentals', 3),
  ('INT 101', 'Programming Fundamentals', 3),
  ('INT 102', 'Web Technology', 1);
  -- ...
```

② Pool — `db/pool.js`

```
import * as mysql from 'mysql2/promise'

export const pool = mysql.createPool({
    host: 'localhost',
    user: 'root',
    password: '143900',
    database: 'sampledb'
})
export default pool;
```

③ Test script — `tests/test-db.js`

```
import pool from "../db/pool.js"

export async function findAll() {
    return await pool.query('SELECT * FROM subjects')   // ได้ [rows, fields]
}
const [subjects, table_schema] = await findAll();
await pool.end();                // ปิด pool ไม่งั้นโปรแกรมไม่จบ
console.log(subjects);
console.log(table_schema);
```

Callback vs Async/Await

```
// Callback
pool.query('SELECT * FROM students WHERE age > ?', [18], (err, results, fields) => {
    if (err) { console.error('Query error:', err); return; }
    console.log('Results:', results);
});

// Async/Await (ใช้แบบนี้)
const [rows] = await pool.query('SELECT * FROM students WHERE age > ?', [18]);
```

④ Repository CRUD — `subject-repository.js`

```
import pool from '../db/pool.js'

export async function findAll() {
    const [subjects] = await pool.query('SELECT * FROM subjects')
    return subjects;
}

export async function findById(id) {
    const [subject] = await pool.query('SELECT * FROM subjects WHERE id = ?', [id])
    if (subject.length == 0) return null;
    return subject;
}

export async function remove(id) {
    const [result] = await pool.query("DELETE FROM subjects WHERE id = ?", [id]);
    return result.affectedRows > 0;
}

export async function create(subject) {
    const [result] = await pool.query(
        "INSERT INTO subjects (subject_code, subject_title, credit) VALUES (?, ?, ?)",
        [subject.code, subject.title, subject.credit]);
    if (result.affectedRows === 0) return false;
    return findById(result.insertId);       // ดึงแถวที่เพิ่งสร้างกลับไป
}

export async function update(subject) {
    const [result] = await pool.query(
        "UPDATE subjects SET subject_code = ?, subject_title = ?, credit = ? WHERE id = ?",
        [subject.code, subject.title, subject.credit, subject.id]);
    if (result.affectedRows === 0) return false;
    return findById(subject.id);
}
```

⑤ **Router: อ่าน Request Body —** `getBody()` (node:http ไม่ parse body ให้ ต้องรวม chunk จาก stream เอง)

```
function getBody(request, response) {
    return new Promise((resolve, reject) => {
        let body = '';
        request.on('data', chunk => { body += chunk.toString(); });   // รับทีละชิ้น
        request.on('end', () => {                                     // รับครบแล้ว
            if (body) resolve(JSON.parse(body));                      // string → object
            else      resolve(null);
        });
        request.on('error', (err) => reject(err));
    });
}
```

⑥ Router: ตอบ JSON — `writeResponse()`

```
async function writeResponse(response, status, message) {
    response.writeHead(status, {'Content-Type': 'application/json'});
    response.end(JSON.stringify(message));
}
```

⑦ Router: POST / PUT

```
async function createNewSubject(request, response) {
    const subject = await getBody(request, response);
    let message = null, status = 201;
    if (subject) {
        const newSubject = await service.createNewSubject(subject)
        message = newSubject
        if (!newSubject) {
            status = 409
            message = {error: `Duplicate subject for id ${subject.id}`}
        }
    } else {
        status = 400
        message = {error: "Bad request, body is empty"}
    }
    writeResponse(response, status, message)
}

async function updateSubject(request, response, id) {
    const subject = await getBody(request, response);
    let message = null, status = 200;
    if (subject) {
        const newSubject = await service.updateSubjectById(id, subject)
        message = newSubject
        if (!newSubject) {
            status = 404
            message = {error: `Subject not found for id ${id}`}
        }
    } else {
        status = 400
        message = {error: "Bad request, body is empty"}
    }
    writeResponse(response, status, message)
}
```

> หมายเหตุ: ในสไลด์เรียก `service.createNewSubject(subject)` โดยไม่มี `await` — เมื่อ repository เป็น async แล้ว **ต้องใส่ `await`** ไม่งั้นได้ Promise แทนข้อมูล

4.4 Flow

```
POST /subjects  body: {"code":"INT 999","title":"Test","credit":3}
  → router        getBody(): on('data') รวม chunk → on('end') JSON.parse
  → body ว่าง?     ใช่ → 400 Bad Request
  → service       createNewSubject(subject)
  → repository    INSERT ... VALUES (?, ?, ?)  ← prepared statement
                  → result.insertId → findById(insertId)
  → router        writeResponse(201, newSubject)
```

---

5. Week 5 — Express.js Framework

5.1 แนวคิด

Express = web framework แบบ minimalist, unopinionated, เร็ว ครอบ `node:http` ไว้ ให้ระบบ **routing** และ **middleware** (ส่วน E ของ MEAN / MERN stack)

Raw node:http vs Express

งาน	node:http	Express
อ่าน body	รวม stream chunk เอง (`getBody`)	`app.use(express.json())` → `req.body`
อ่าน URL/query	`new URL()` + `searchParams`	`req.query` , `req.params`
Routing	if/else + switch เอง	`router.get('/:id', ...)`
ตอบกลับ	`writeHead` + `end(JSON.stringify())`	`res.status(201).json(obj)`

Request parser - `req.params` — ค่าจาก path (`/api/subjects/:id` → `req.params.id`) - `req.query` — ค่าจาก query string (`?page=2&size=10`) - `req.body` — body ที่ parse แล้ว (ต้องมี `express.json()` middleware)

Response - `res.json(obj)` — แปลง object → JSON ส่งกลับ (status 200) - `res.status(code)` — ตั้ง status **แต่ยังไม่ส่ง** ต้อง chain ต่อ `.json()` / `.send()` / `.end()` - `res.status(204).end()` — ตอบไม่มี body

Middleware = ฟังก์ชัน `(req, res, next)` ทำงานก่อนถึง route (parse body, auth, log, CORS, error) - เรียก `next()` → ไป middleware ตัวถัดไป - เรียก `res.send()/json()/end()` → จบ ตอบ client - ประเภท: Application-level, Router-level, Error-handling, Built-in, Third-party

หน้าที่ของแต่ละ Layer (ออกสอบ)

Layer	หน้าที่	ควรมี	ห้ามมี	เปรียบเหมือน
Controller / Router	รับ HTTP request/response	routing, mapping, validation	business logic, DB code	แผนกต้อนรับ รพ.
Service	กฎทางธุรกิจ ประสานงาน	business logic, calculation	รายละเอียด HTTP/DB	หมอ (วินิจฉัย)
Repository	เข้าถึงข้อมูล CRUD	queries, persistence	business logic, HTTP	ห้องเวชระเบียน

5.2 ต้องทำอะไร (สร้าง Template Express-ESM)

1. สร้างโปรเจกต์ `npm init -y`
2. `npm install express mysql2` (+ `nodemon` สำหรับ auto-restart)
3. เพิ่ม `"type": "module"` ใน `package.json`
4. สร้างโฟลเดอร์ `src/configs` , `src/repositories` , `src/services` , `src/routes`
5. สร้าง `db-config.js` → repository → service → route → `app.js`
6. (WebStorm) Tools → Save project as template: `Express-ESM`

5.3 โค้ด

① Express พื้นฐาน — `test-express/src/app.js`

```
import express from 'express';

const app = express();
app.use(express.json());                    // middleware: parse JSON body

app.put('/api/subjects/:id', (req, res) => {
    const {filter, page, size} = req.query;  // ?filter=..&page=..&size=..
    const id = req.params.id;                // :id
    const subject = req.body;                // JSON body
    subject.id = id;
    res.json([subject, {filter, page, size}]);
});

app.post('/api/subjects/:id', (req, res) => {
    const subject = req.body;
    subject.id = req.params.id;
    res.json(subject);
});

app.listen(3000);
```

② DB Config — `src/configs/db-config.js`

```
import mysql from 'mysql2/promise'

const db = mysql.createPool({
    host: 'localhost',
    user: 'root',
    password: '',
    database: 'classicmodels',   // หรือ sakila / mobile_distributor
})
export default db
```

③ Repository (generic template) — `src/repositories/example-repo.js`

> เทคนิค: ประกาศ SQL เป็นค่าคงที่ด้านบน แล้วเปลี่ยนแค่ `TABLE_NAME` / ชื่อคอลัมน์ PK ก็ใช้กับตารางอื่นได้ทันที `SET ?` + ส่ง object → mysql2 แปลงเป็น `col1 = val1, col2 = val2` ให้อัตโนมัติ

```
import db from '../configs/db-config.js'

const TABLE_NAME = "subjects"
const ENTITY_NAME = "Subject"

const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`
const FIND_ONE_SQL = `SELECT * FROM ${TABLE_NAME} WHERE id = ?`
const UPDATE_SQL   = `UPDATE ${TABLE_NAME} SET ? WHERE id = ?`
const REMOVE_SQL   = `DELETE FROM ${TABLE_NAME} WHERE id = ?`
const CREATE_SQL   = `INSERT INTO ${TABLE_NAME} SET ?`

export async function findAll() {
    const [datas] = await db.query(FIND_ALL_SQL)
    return datas
}

export async function findOne(id) {
    const [data] = await db.query(FIND_ONE_SQL, [id])
    if (data.length === 0) {
        const err = new Error(`${ENTITY_NAME} not found for id = ${id}`)
        err.statusCode = 404
        err.code = "NOT FOUND"
        throw err                       // โยนให้ error handler จัดการ
    }
    return data[0]
}

export async function update(id, data) {
    await findOne(id)                   // เช็คว่ามีอยู่จริงก่อน (ไม่มี → 404)
    await db.query(UPDATE_SQL, [data, id])
    return findOne(id)                  // คืนค่าใหม่หลังแก้
}

export async function remove(id) {
    await findOne(id)
    const [data] = await db.query(REMOVE_SQL, [id])
    return data
}

export async function create(data) {
    const [result] = await db.query(CREATE_SQL, [data])
    return findOne(result.insertId)
}
```

④ Service — `src/services/example-service.js`

```
import * as repo from "../repositories/example-repo.js"

export async function getObject(id)            { return await repo.findOne(id) }
export async function getAllObjects()          { return await repo.findAll() }
export async function updateObject(id, data)   { return await repo.update(id, data) }
export async function removeObject(id)         { return await repo.remove(id) }
export async function createObject(data)       { return await repo.create(data) }
```

⑤ Route — `src/routes/example-route.js`

```
import * as service from "../services/example-service.js"
import express from "express"

const router = express.Router()

router.get("/", async (req, res) => {
    res.json(await service.getAllObjects())                // 200
})
router.get("/:id", async (req, res) => {
    res.json(await service.getObject(req.params.id))       // 200 / 404
})
router.post("/", async (req, res) => {
    const result = await service.createObject(req.body)
    res.status(201).json(result)                           // 201 Created
})
router.put("/:id", async (req, res) => {
    res.json(await service.updateObject(req.params.id, req.body))  // 200
})
router.delete("/:id", async (req, res) => {
    await service.removeObject(req.params.id)
    res.status(204).end()                                  // 204 No Content
})

export default router
```

⑥ App — `src/app.js`

```
import express from 'express'
import exRouter from './routes/example-route.js'

const app = express()
app.use(express.json())              // 1) parse body ก่อนเสมอ
app.use('/examples', exRouter)       // 2) ผูก router กับ path prefix

app.listen(3000, () => console.log('Example app listening on port 3000'))
```

⑦ ทดสอบ service โดยตรง — `src/test-service.js`

```
import * as service from './services/example-service.js';
const data = { subject_code: 'INT 099', subject_title: 'IT Fundamentals', credit: 3 }
// console.log(await service.createObject(data))
// console.log(await service.updateObject(26, data))
await service.removeObject(26)
console.log(await service.getObject(26))   // ควรได้ error 404
```

> ตัวอย่างเดียวกันใช้กับตาราง `actor` ของฐาน `sakila` ใน `express-template2/` — เปลี่ยนแค่ `TABLE_NAME = "actor"` และ PK เป็น `actor_id` , mount ที่ `/actors`

5.4 Flow

```
GET /examples/5
  → app.js        express.json()  (GET ไม่มี body ผ่านไป)
  → app.js        '/examples' match → ส่งต่อให้ exRouter ด้วย path '/5'
  → route         router.get('/:id') → req.params.id = "5"
  → service       getObject("5")
  → repository    SELECT * FROM subjects WHERE id = ?  [5]
                  เจอ → return data[0]   |  ไม่เจอ → throw err (statusCode 404)
  → route         res.json(object)  → 200
```

---

6. Week 6 — Error Handling

6.1 แนวคิด

- **Exception/Error** = เหตุการณ์ที่ทำให้โปรแกรมไม่ไหลตามปกติ
- ถ้า throw แล้ว **ไม่มีใคร catch** → Node.js `process.exit(1)` → **server ล่มทั้งตัว**
- Error เป็น **object**

ทำไมต้องจัดการ Error: กัน app crash, UX ดีขึ้น (ข้อความเป็นมิตร), **ไม่ให้ stack trace รั่ว**, ช่วย debug/monitor

ประเภท Error - **Programming Errors** — bug, logic ผิด (TypeError) - **Operational Errors** — network timeout, DB ล่ม, ไฟล์ไม่เจอ - **Expected vs Unexpected** — ที่คาดได้ต้องจัดการให้ดี

Error Propagation — error ถูกส่ง **ขึ้นไปตาม call stack** จนกว่าจะเจอ `catch` ; ถ้าไม่เจอเลยถึง global scope → โปรแกรมตาย

```
main() → a() → b()  ← throw ที่นี่
                ↑ ไม่ catch → ขึ้นไป a() → ขึ้นไป main() → try/catch ข้างนอกจับได้
```

Property ของ Error object - `message` — ข้อความ - `name` — ชนิด (`Error` , `TypeError` , `ReferenceError`) - `stack` — stack trace - (system error ของ Node) `code` เช่น `ENOENT` , `EACCES` / `errno`

Error ใน Express 1. Error เกิด (throw, Promise reject, DB error) 2. ส่งต่อด้วย `next(err)` → Express **ข้าม middleware/route ปกติทั้งหมด** ไปที่ error handler 3. **Error-handling middleware มี 4 parameter:** `(err, req, res, next)` (ปกติมี 3) 4. ตอบ status code + JSON 5. **ต้องประกาศไว้ท้ายสุด** หลัง route ทั้งหมด (ถ้าไม่มี Express จะตอบ HTML default)

> **Express 5** (ที่ใช้ในโปรเจกต์ `"express": "^5.2.1"`) — ถ้า async route handler throw/reject, Express ส่งไป error handler ให้อัตโนมัติ **ไม่ต้องเขียน try/catch + next(err)** เอง (Express 4 ต้องเขียนเอง)

Status Codes ที่ต้องจำ

Code	ความหมาย	ใช้เมื่อ
200 OK	สำเร็จ	GET, PUT
201 Created	สร้างสำเร็จ	POST
204 No Content	สำเร็จ ไม่มี body	DELETE
400 Bad Request	request ผิด (ขาด body/param)	validation fail, FK ไม่มีจริง
401 Unauthorized	ยังไม่ login / auth fail	
403 Forbidden	login แล้วแต่ไม่มีสิทธิ์	
404 Not Found	ไม่เจอ resource	id ไม่มี
409 Conflict	ขัดกับ state ปัจจุบัน	ข้อมูลซ้ำ (duplicate key), ลบแล้วติด FK
412 Precondition Failed	เงื่อนไขใน header เป็นเท็จ	
500 Internal Server Error	error ทั่วไปฝั่ง server	
503 Service Unavailable	service ไม่พร้อม	

6.2 ต้องทำอะไร

1. เข้าใจ try/catch + propagation (ลอง `test-exception-handling.js`)
2. สร้าง **custom error class** `AppError` (มี `statusCode`)
3. สร้าง **error-handling middleware** `src/middlewares/error-handler.js`
4. ใน repository: เมื่อไม่เจอข้อมูล → `throw new AppError(msg, 404)`
5. ใน error handler: แปลง **MySQL error code** → HTTP status (ER_DUP_ENTRY → 409 ฯลฯ)
6. `app.use(errorHandler)` **บรรทัดสุดท้าย** หลังผูก router ทั้งหมด

6.3 โค้ด

① try/catch + propagation (sync)

```
import fs from 'node:fs';

function main(value = 0) { console.log(a(value)); }
function a(value) { return b(value); }
function b(value) {
    if (value > 0) return ++value;
    const data = fs.readFileSync('test.txt');   // ไม่มีไฟล์ → throw ENOENT
}

try {
    main(1);                        // ok → พิมพ์ 2
    console.log('After main success');
    main();                         // throw → กระโดดไป catch
    console.log('After main error');   // ไม่ถูกรัน
} catch (e) {
    console.log('Message: ', e.message);
    console.log('Code: ', e.code);       // ENOENT
    console.log('Stack Trace: ', e.stack);
}
console.log('Program was normal ended');  // โปรแกรมไม่ตาย
```

② Custom Error (เพิ่ม property เอง)

```
async function b(value) {
    if (value > 0) return ++value;
    try {
        return await fs.readFile('data.txt', 'utf8');
    } catch (e) {
        const err = new Error('File data.txt not found');
        err.code = 'FILE_NOT_FOUND';
        err.status = 404;
        throw err;           // โยน error ที่มีข้อมูลมากขึ้นออกไป
    }
}
```

③ Custom Error Class (ตามสไลด์ — `test-exception-handling.js`)

```
class AppError extends Error {
    errors = {404:'NOT_FOUND', 400:'BAD REQUEST', 409:'CONFLICT',
              500:'INTERNAL SERVER ERROR', 403:'FORBIDDEN', 401:'UNAUTHORIZED'}
    constructor(message, statusCode) {
        super(message);
        this.statusCode = statusCode;
        this.error = this.errors[statusCode];
        this.isOperational = true;                    // error ที่คาดไว้
        Error.captureStackTrace(this, this.constructor);
    }
}
// ใช้: catch (e) { const err = new AppError(e.message, 404); console.log(err) }
```

④ Middleware พื้นฐาน (logger)

```
// Middleware 1 – Logger
app.use((req, res, next) => {
    console.log(`${req.method} ${req.url}`)
    next()                                  // ส่งต่อ
})
// Middleware 2 – JSON parser
app.use(express.json())
app.use('/films', filmRouter);
```

⑤ Error Handler แบบสไลด์ (เรียบง่าย)

```
app.use(function (err, req, res, next) {      // 4 params!
    const status = err.status || 500;
    res.status(status).json({
        error: err.code,
        statusCode: status,
        message: err.message,
        path: req.originalUrl,
        timestamp: new Date().toLocaleString()
    });
});
```

⑥ Error Handler ตัวเต็ม — `src/middlewares/error-handler.js` (เวอร์ชันใน `node-express-esm-template`)

```
import httpStatus from "http-status";       // npm install http-status
Error.stackTraceLimit = 1                    // ตัด stack trace ให้สั้น

export class AppError extends Error {
    constructor(message, statusCode) {
        super(message);
        this.statusCode = statusCode;
        this.errorCode = httpStatus[statusCode];   // 404 → "Not Found"
        Error.captureStackTrace(this, this.constructor);
    }
}

export const errorHandler = (err, req, res, next) => {
    console.log(err);
    let statusCode = err.statusCode || 500;
    let errorCode  = err.errorCode  || 'INTERNAL_SERVER_ERROR';
    let message    = err.message    || 'An unexpected error occurred';

    // แปลง MySQL Error Code → HTTP status ที่เหมาะสม
    if (err.code === 'ER_DUP_ENTRY') {                // ค่า unique/PK ซ้ำ
        statusCode = 409;
        errorCode = 'DUPLICATE_KEY';
        message = 'Resource already exists (Primary key or unique constraint violation)';
    } else if (err.code === 'ER_NO_REFERENCED_ROW_2') { // FK ที่อ้างถึงไม่มีจริง
        statusCode = 400;
        errorCode = 'FOREIGN_KEY_NOT_FOUND';
        message = 'Referenced foreign key record does not exist';
    } else if (err.code === 'ER_ROW_IS_REFERENCED_2') { // ลบแถวที่มีคนอ้างอยู่
        statusCode = 409;
        errorCode = 'RESOURCE_IN_USE';
        message = 'Cannot delete or modify resource because it is referenced by related
records';
    }

    res.status(statusCode).json({
        success: false,
        error: { code: errorCode, message: message },
        timestamp: new Date().toISOString()
    });
};
```

⑦ ใช้ใน Repository

```
import {AppError} from "../middlewares/error-handler.js";
import httpStatus from "http-status";

export async function findOne(id) {
    const [row] = await db.query(FIND_ONE_SQL, [id])
    if (row.length === 0) {
        throw new AppError(`${ENTITY_NAME} with code '${id}' was not found`,
                           httpStatus.NOT_FOUND)          // 404
    }
    return row[0]
}
```

⑧ ผูกใน app.js (ต้องอยู่ท้ายสุด)

```
app.use(express.json())
app.use('/api/offices', officeRouter)
app.use(errorHandler)        // ← หลัง router ทุกตัว
```

6.4 Flow

```
GET /api/offices/999
  → route         service.getObject("999")
  → repository    SELECT ... → 0 แถว → throw new AppError("...not found", 404)
  → (error ไหลย้อน) service → route (async reject)
  → Express 5     จับ rejected promise → next(err) ให้อัตโนมัติ
  → ข้าม route/middleware ปกติทั้งหมด
  → errorHandler(err, req, res, next)
                  statusCode = 404 → res.status(404).json({success:false, error:{...}})

POST /api/products  (sku ซ้ำ)
  → repository    INSERT → MySQL throw { code: 'ER_DUP_ENTRY' }
  → errorHandler  แปลงเป็น 409 DUPLICATE_KEY
```

---

7. Week 7 — Pagination & DTO

7.1 แนวคิด Pagination

ทำไมต้องแบ่งหน้า: คืนทุกแถว → response ใหญ่, DB ทำงานหนัก, ใช้ memory/network มาก → pagination ทำให้ขนาด response คาดเดาได้

พารามิเตอร์: `GET /api/products?page=2&limit=20` - `page` — หน้าที่ต้องการ - `limit` — จำนวนต่อหน้า - `offset` — จำนวนแถวที่ข้าม (**คำนวณเอง**)

> **`OFFSET = (page − 1) × limit`**

page	limit	offset	ได้แถวที่
1	10	0	1–10
2	10	10	11–20
3	10	20	21–30
9	10	80	81–90

SQL: `SELECT * FROM products LIMIT ? OFFSET ?` + `SELECT COUNT(*) as total FROM products`

Metadata ที่คำนวณ - `totalPages = Math.ceil(total / limit)` - `hasNextPage = page * limit < total` - `hasPrevPage = page > 1`

Best Practices - `page` ต้อง ≥ 1 → `Math.max(parseInt(page), 1)` - กำหนด limit สูงสุด เช่น 100 → `Math.min(parseInt(limit), 100)` - ใช้ prepared statement - สร้าง index ให้คอลัมน์ที่ filter/sort - หน้าลึกมาก ๆ พิจารณา **cursor pagination**

7.2 แนวคิด DTO (Data Transfer Object)

- **DTO** = object ที่กำหนด **รูปร่างข้อมูล** ที่แลกเปลี่ยนระหว่าง layer / ส่งออก API
- แยก **API contract** ออกจาก **database model**
- ประโยชน์: **ไม่ให้ข้อมูลอ่อนไหวรั่ว**, response สม่ำเสมอ, validate/document ง่าย, ลด coupling ระหว่าง DB กับ client
- **Mapper** แปลง Entity → DTO ได้หลายแบบ (เช่น Summary / Detail)

```
Repository ──Entity──► Service ──mapper──► DTO ──► Controller ──► Front-end
```

Entity vs DTO

	ฟิลด์
Database Entity (users)	id, name, email, **password_hash**, role, created_at
User Response DTO	id, name, email, role

→ ห้ามส่ง `password_hash` ออกไปเด็ดขาด

รูปแบบ JSON Response มาตรฐาน

```
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "total": 138, "page": 2, "limit": 20,
    "totalPages": 7, "hasNextPage": true, "hasPrevPage": true
  },
  "timestamp": "2026-10-08T15:00:00.000Z"
}
```

7.3 ต้องทำอะไร (ลำดับ)

1. `npm install http-status`
2. สร้าง **pagination middleware** (`src/middlewares/pagination-middleware.js`) → ใส่ `req.pagination = {page, limit, offset}` หรือ `null`
3. `app.use(queryParser())` **ก่อน** router
4. Repository: `findAll(pagination)` — ถ้าไม่มี pagination คืนทั้งหมด, ถ้ามีใช้ `LIMIT ? OFFSET ?` + แนบ `rows.totalItems = await count()`
5. สร้าง **mapper** (`src/mappers/product-mapper.js`) — `toSummaryDTO` , `toDetailDTO` , `map` , `maps`
6. Service: เรียก repo แล้ว map เป็น DTO
7. สร้าง **apiResponse** (`src/mappers/api-response.js`) — ห่อ `{success, data, pagination?, timestamp}`
8. Route: อ่าน `req.pagination` , `req.query.view` → เรียก service → `apiResponse(res, data, pagination)`

7.4 โค้ด

① Pagination Middleware — `src/middlewares/pagination-middleware.js`

```
export const queryParser = () => {
    return (req, res, next) => {
        const query = req.query || {};
        // ไม่ส่ง page หรือ limit มา → ไม่แบ่งหน้า
        if (query.page == undefined || query.limit == undefined) {
            req.pagination = null
            return next();
        }
        let page = query.page || 1                      // ว่าง → 1
        const pageSize = query.limit || 20              // ว่าง → 20
        page = Math.max(parseInt(page), 1)              // ติดลบ → 1
        const limit = Math.min(parseInt(pageSize), 100)  // เกิน 100 → 100
        const offset = (page - 1) * limit               // ⚠ ใช้ limit (ในโค้ดเดิมใช้ pageSize)
        req.pagination = {page, limit, offset}
        next();
    }
}
```

② app.js — ลงทะเบียน middleware

```
import {queryParser} from "./middlewares/pagination-middleware.js";

const app = express()
app.use(express.json())
app.use(queryParser())          // ต้องอยู่ก่อน router

// ทดสอบ: GET /?page=2&limit=10 → {"page":2,"limit":10,"offset":10}
app.get('/', (req, res) => res.json(req.pagination))
```

③ Repository (มี count + LIMIT/OFFSET) — `src/repositories/product-repo.js`

```
import db from '../configs/db-config.js'
import {AppError} from "../middlewares/error-handler.js";
import httpStatus from "http-status";

const TABLE_NAME = "products"
const ENTITY_NAME = "Product"

const COUNT_ALL    = `SELECT COUNT(*) as total FROM ${TABLE_NAME}`
const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`
const FIND_ONE_SQL = `SELECT * FROM ${TABLE_NAME} WHERE id = ?`
const UPDATE_SQL   = `UPDATE ${TABLE_NAME} SET ? WHERE id = ?`
const REMOVE_SQL   = `DELETE FROM ${TABLE_NAME} WHERE id = ?`
const CREATE_SQL   = `INSERT INTO ${TABLE_NAME} SET ?`

async function count() {
    const [[{ total }]] = await db.query(COUNT_ALL);   // destructure: [rows][0].total
    return total;
}

export async function findAll(pagination) {
    if (!pagination) {
        const [rows] = await db.query(FIND_ALL_SQL)
        return rows
    }
    const {limit, offset} = pagination
    const [rows] = await db.query(FIND_ALL_SQL + ' LIMIT ? OFFSET ?', [limit, offset])
    rows.totalItems = await count()      // แนบจำนวนทั้งหมดไว้กับ array
    return rows
}

export async function findOne(id) {
    const [row] = await db.query(FIND_ONE_SQL, [id])
    if (row.length === 0) {
        throw new AppError(`${ENTITY_NAME} with id '${id}' was not found`, httpStatus.NOT_FOUND)
    }
    return row[0]
}

export async function create(data) {
    const [result] = await db.query(CREATE_SQL, [data])   // sku ซ้ำ → ER_DUP_ENTRY → 409
    return findOne(result.insertId)
}

export async function update(id, data) {
```

```
    await findOne(id)
    await db.query(UPDATE_SQL, [data, id])
    return findOne(id)
}

export async function remove(id) {
    await findOne(id)
    const [data] = await db.query(REMOVE_SQL, [id])
    return data
}
```

> `[[{ total }]]` อ่านว่า: ผลลัพธ์ query = `[rows, fields]` → เอา `rows` → เอาแถวแรก → เอา property `total`

④ Mapper (DTO) — `src/mappers/product-mapper.js`

ตัวอย่างในสไลด์ (ฐาน `classicmodels`):

```
export default class ProductMapper {
    // Summary DTO: ส่งเฉพาะฟิลด์สำคัญ (ใช้กับ list)
    static toSummaryDTO(entity) {
        return {
            id: entity.productCode,
            name: entity.productName,
            scale: entity.productScale,
            retailPrice: entity.MSRP,
        };
    }
    // Detail DTO: ทุกฟิลด์ แต่เปลี่ยนชื่อ / ซ่อนบางตัว
    static toDetailDTO(entity) {
        const {MSRP, buyPrice, ...data} = entity   // ดึง 2 ฟิลด์ออก ที่เหลือเก็บใน data
        data.retailPrice = MSRP
        data.price = buyPrice
        return data
    }
    // แปลง 1 ตัว
    static map(entity, viewType) {
        return viewType == 'summary' ? this.toSummaryDTO(entity) : this.toDetailDTO(entity)
    }
    // แปลงทั้ง list (ต้องเก็บ totalItems ไว้ด้วย เพราะ .map() สร้าง array ใหม่ทำให้หาย)
    static maps(entities, viewType) {
        const {totalItems} = entities
        const entitiesMap = viewType == 'full'
            ? entities.map(item => this.toDetailDTO(item))
            : entities.map(item => this.toSummaryDTO(item))
        if (totalItems) entitiesMap.totalItems = totalItems
        return entitiesMap
    }
}
```

สำหรับฐาน **`mobile_distributor`** (โจทย์สอบ — test ต้องการ summary มี `id`, `sku`, `name`):

```
static toSummaryDTO(entity) {
    return {
        id: entity.id,
        sku: entity.sku,
        name: entity.name,
        brandId: entity.brand_id,
        category: entity.category,
        price: entity.price,
        status: entity.status,
    };
}
static toDetailDTO(entity) {
    const {brand_id, stock_quantity, ram_gb, storage_gb, release_date, ...data} = entity
    return { ...data, brandId: brand_id, stockQuantity: stock_quantity,
             ramGb: ram_gb, storageGb: storage_gb, releaseDate: release_date }
}
```

⑤ Service — `src/services/product-service.js`

```
import * as repo from "../repositories/product-repo.js"
import mapper from "../mappers/product-mapper.js"

export async function getObject(id, viewType) {
    return mapper.map(await repo.findOne(id), viewType)
}
export async function getAllObjects(pagination, viewType) {
    const objects = await repo.findAll(pagination)
    return mapper.maps(objects, viewType)
}
export async function createObject(data)      { return await repo.create(data) }
export async function updateObject(id, data)  { return await repo.update(id, data) }
export async function removeObject(id)        { return await repo.remove(id) }
```

⑥ API Response — `src/mappers/api-response.js`

```
import httpStatus from "http-status";

export async function apiResponse(res, data, pagination = null, status = httpStatus.OK) {
    return pagination
        ? paginate(res, data, pagination, status)
        : success(res, data, status)
}

// Response ทั่วไป
async function success(res, data, statusCode) {
    return res.status(statusCode).json({
        success: true,
        data: data,
        timestamp: new Date().toISOString(),
    });
}

// Response แบบแบ่งหน้า
async function paginate(res, data, pagination, statusCode) {
    const {limit, page} = pagination
    const total = data.totalItems;
    return res.status(statusCode).json({
        success: true,
        data,
        pagination: {
            total: total,
            page: page,
            limit: limit,
            totalPages: Math.ceil(total / limit),
            hasNextPage: page * limit < total,
            hasPrevPage: page > 1
        },
        timestamp: new Date().toISOString()
    });
}
```

> ระวัง: ถ้าจะส่ง status อื่น (เช่น 201) ต้องส่งเป็น **argument ตัวที่ 4** → `apiResponse(res, data, null, httpStatus.CREATED)`

⑦ Route — `src/routes/product-route.js`

```
import * as service from "../services/product-service.js"
import express from "express"
import httpStatus from "http-status";
import {apiResponse} from "../mappers/api-response.js";

const router = express.Router()

router.get("/", async (req, res) => {
    const pagination = req.pagination                // จาก middleware
    const viewType = req.query.view || "summary"     // list → summary เป็นค่าเริ่มต้น
    const objects = await service.getAllObjects(pagination, viewType)
    await apiResponse(res, objects, pagination)
})

router.get("/:id", async (req, res) => {
    const viewType = req.query.view || "full"        // รายตัว → full เป็นค่าเริ่มต้น
    const object = await service.getObject(req.params.id, viewType)
    await apiResponse(res, object)
})

router.post("/", async (req, res) => {
    const result = await service.createObject(req.body)
    await apiResponse(res, result, null, httpStatus.CREATED)      // 201
})

router.put("/:id", async (req, res) => {
    const result = await service.updateObject(req.params.id, req.body)
    await apiResponse(res, result)                                // 200
})

router.delete("/:id", async (req, res) => {
    await service.removeObject(req.params.id)
    res.status(httpStatus.NO_CONTENT).end()                       // 204
})

export default router
```

7.5 Flow

```
GET /api/67130500001/products?page=2&limit=10&view=summary
  → express.json()
  → queryParser()     page=2, limit=10 → offset=(2-1)*10=10
                      req.pagination = {page:2, limit:10, offset:10}
  → productRouter     router.get('/')
                      viewType = "summary"
  → service           getAllObjects(pagination, "summary")
  → repository        SELECT * FROM products LIMIT 10 OFFSET 10
                      rows.totalItems = SELECT COUNT(*) → 60
  → mapper.maps       entity[] → SummaryDTO[] (+ คง totalItems)
  → apiResponse       มี pagination → paginate()
                      totalPages=ceil(60/10)=6, hasNextPage=20<60=true, hasPrevPage=true
  → res 200           {success, data, pagination, timestamp}
```

8. สร้างโปรเจกต์ตั้งแต่ศูนย์ (Step-by-step ครบทุกไฟล์)

นี่คือ **ลำดับที่ควรทำในห้องสอบ** (รวมทุกอย่างจาก Week 4–7)

Step 0 — โครงสร้างไฟล์สุดท้าย

```
project/
├── package.json                  ← "type": "module", script start
└── src/
    ├── app.js                    ← ประกอบทุกอย่าง (ทำสุดท้าย)
    ├── configs/
    │   └── db-config.js          ← Step 2
    ├── middlewares/
    │   ├── error-handler.js      ← Step 3
    │   └── pagination-middleware.js ← Step 4
    ├── repositories/
    │   ├── brand-repo.js         ← Step 5
    │   └── product-repo.js
    ├── mappers/
    │   ├── api-response.js       ← Step 6
    │   └── product-mapper.js     ← Step 7
    ├── services/
    │   ├── brand-service.js      ← Step 8
    │   └── product-service.js
    └── routes/
        ├── brand-route.js        ← Step 9
        └── product-route.js
```

ลำดับการเขียน = จากล่าง (DB) ขึ้นบน (HTTP) เพราะแต่ละชั้น import ชั้นที่อยู่ล่างกว่า: `config → middleware (AppError) → repository → mapper → service → route → app.js`

Step 1 — Setup โปรเจกต์

```
mkdir project && cd project
npm init -y
npm install express mysql2 http-status
npm install -D nodemon
mkdir -p src/{configs,middlewares,repositories,mappers,services,routes}
```

`package.json` (ส่วนที่ต้องแก้):

```
{
  "type": "module",
  "main": "src/app.js",
  "scripts": {
    "start": "nodemon src/app.js"
  }
}
```

Step 2 — `src/configs/db-config.js`

```
import mysql from 'mysql2/promise'

const db = mysql.createPool({
    host: "localhost",
    user: "root",
    password: "",
    database: "mobile_distributor",
});
export default db
```

✅ **เช็ค**: ลอง `node -e "import('./src/configs/db-config.js').then(async m=>{console.log((await m.default.query('select 1'))[0]);process.exit()})"`

Step 3 — `src/middlewares/error-handler.js`

ใช้โค้ดจาก 6.3 ⑥ (AppError + errorHandler)

Step 4 — `src/middlewares/pagination-middleware.js`

ใช้โค้ดจาก 7.4 ①

Step 5 — Repositories

- `product-repo.js` → ใช้โค้ดจาก 7.4 ③
- `brand-repo.js` → **copy product-repo แล้วเปลี่ยนแค่ 2 บรรทัด**:

```
const TABLE_NAME = "brands"
const ENTITY_NAME = "Brand"
```

Step 6 — `src/mappers/api-response.js`

ใช้โค้ดจาก 7.4 ⑥

Step 7 — `src/mappers/product-mapper.js`

ใช้โค้ดจาก 7.4 ④ (เวอร์ชัน mobile_distributor)

Step 8 — Services

- `product-service.js` → 7.4 ⑤
- `brand-service.js` (ไม่มี mapper):

```
import * as repo from "../repositories/brand-repo.js"

export async function getObject(id)           { return await repo.findOne(id) }
export async function getAllObjects(pagination){ return await repo.findAll(pagination) }
export async function createObject(data)      { return await repo.create(data) }
export async function updateObject(id, data)  { return await repo.update(id, data) }
export async function removeObject(id)        { return await repo.remove(id) }
```

Step 9 — Routes

- `product-route.js` → 7.4 ⑦
- `brand-route.js` :

```
import * as service from "../services/brand-service.js";
import express from "express";
import { apiResponse } from "../mappers/api-response.js";

const router = express.Router()

router.get("/", async (req, res) => {
    const {pagination} = req;
    await apiResponse(res, await service.getAllObjects(pagination), pagination)
})
router.get("/:id", async (req, res) => {
    await apiResponse(res, await service.getObject(req.params.id))
})

export default router;
```

Step 10 — `src/app.js` (ประกอบร่าง — ลำดับ app.use สำคัญมาก)

```
import express from 'express'
import brandRouter from "./routes/brand-route.js"
import productRouter from "./routes/product-route.js"
import {errorHandler} from "./middlewares/error-handler.js";
import {queryParser} from "./middlewares/pagination-middleware.js";

const STUDENT_ID = '67130500001'
const app = express()

// ① middleware ทั่วไป (ก่อน router)
app.use(express.json())
app.use(queryParser())

// ② routes
app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
    res.json({
        success: true,
        data: { studentId: STUDENT_ID, name: "ชื่อ-นามสกุล", secretKey: "xxxx" },
        timestamp: new Date().toISOString(),
    })
})
app.use(`/api/${STUDENT_ID}/brands`, brandRouter)
app.use(`/api/${STUDENT_ID}/products`, productRouter)

// ③ error handler (ท้ายสุดเสมอ)
app.use(errorHandler)

app.listen(3000, () => console.log('Example app listening on port 3000'))
```

Step 11 — รัน & ทดสอบ

```
npm start
# อีก terminal
curl -i http://localhost:3000/api/67130500001/brands
curl -i "http://localhost:3000/api/67130500001/products?page=1&limit=10"
curl -i http://localhost:3000/api/67130500001/products/1
curl -i http://localhost:3000/api/67130500001/products/99999          # 404
curl -i -X POST http://localhost:3000/api/67130500001/products \
     -H "Content-Type: application/json" \
     -d '{"sku":"TEST-
1","name":"Test","brand_id":1,"category":"FLAGSHIP","price":100,"status":"AVAILABLE"}'   # 201
curl -i -X PUT http://localhost:3000/api/67130500001/products/<id> \
     -H "Content-Type: application/json" -d '{"price":200}'           # 200
curl -i -X DELETE http://localhost:3000/api/67130500001/products/<id> # 204

# ตัวตรวจอัตโนมัติ
node tests/test-runner.mjs 67130500001
```

9. Flow ของ 1 Request ตั้งแต่เข้าจนออก



สรุปว่าใครส่ง status อะไร

สถานการณ์	ใครเป็นคนกำหนด	Status
GET สำเร็จ	apiResponse (default)	200
POST สำเร็จ	route ส่ง `httpStatus.CREATED`	201
PUT สำเร็จ	apiResponse	200
DELETE สำเร็จ	route `res.status(204).end()`	204
ไม่เจอ id	repository `throw AppError(404)` → errorHandler	404
ค่า unique ซ้ำ	MySQL `ER_DUP_ENTRY` → errorHandler	409
FK ไม่มีจริง	MySQL `ER_NO_REFERENCED_ROW_2` → errorHandler	400
ลบแถวที่ถูกอ้างอิง	MySQL `ER_ROW_IS_REFERENCED_2` → errorHandler	409
error อื่น ๆ	errorHandler default	500

10. Checklist สอบปฏิบัติ (จาก `tests/test-runner.mjs`)

Base URL: `http://localhost:3000/api/<studentId>`

#	Test	คะแนน	สิ่งที่ต้องได้
1	`GET /student-info`	10	200, `data.studentId` และ `data.secretKey` ตรง
2	`GET /brands`	5	200, `data` เป็น array ≥ 5 ตัว, มี `id` และ `name`
3	`GET /brands/1` และ `/brands/99999`	5	200 / **404**
4	`GET /products?page=1&limit=10`	15	`success:true`, มี `pagination` (`page`, `limit`, `total` ≥50, `totalPages` ≥5, `hasNextPage:true`)
5	Summary DTO (`GET /products?page=1&limit=5`)	10	item มี **`id`, `sku`, `name`**
6	`GET /products/1` และ `/products/99999`	10	200 (มี `id`, `sku`, `name`) / **404**
7	`POST /products`	10	**201**, response มี `data.id` (หรือ `result.id`)
8	`PUT /products/:id`	10	**200**
9	`DELETE /products/:id`	10	**204** (หรือ 200) แล้ว GET ซ้ำต้อง **404**
10	POST sku ซ้ำ	5	**409** (หรือ 400) และ `success:false`
11	Envelope	5	มี `success` และ `timestamp`
12	Route พหูพจน์	5	`/products` ต้องได้ 200

ผลล่าสุดใน `test-report-67130500001.json` = **60/100** — ตก test 5, 7, 8, 9 (สาเหตุดูข้อ 11)

11. บั๊ก/จุดผิดที่เจอในโค้ดปัจจุบัน

ไฟล์	ปัญหา	วิธีแก้
`node-express-esm-template/src/mappers/product-mapper.js`	ใช้ฟิลด์ของ `classicmodels` (`productCode`, `productName`, `MSRP`) แต่ DB เป็น `mobile_distributor` (`id`, `sku`, `name`) → summary ไม่มี `id/sku/name` → **ตก test 5**	เปลี่ยน `toSummaryDTO` / `toDetailDTO` ตาม 7.4 ④
`node-express-esm-template/src/repositories/product-repo.js` → `create()`	copy มาจาก productline: ใช้ `data.productLine` เป็น key → `findOne(undefined)` → throw 404 → **POST ได้ 404 แทน 201** → test 7, 8, 9 ตกตาม	`const [result] = await db.query(CREATE_SQL, [data]); return findOne(result.insertId)`
`node-express-esm-template/src/services/product-service.js` → `getObject`	ไม่ส่ง `viewType` ให้ `mapper.map`	`mapper.map(await repo.findOne(id), viewType)`
`node-express-esm-template/src/routes/office-route.js`	พิมพ์ผิด `apiRespone`, `apiRespone.success` ไม่มีอยู่จริง; ส่ง `httpStatus.CREATED` เป็น arg ที่ 3 (ช่อง pagination)	ใช้ `apiResponse(res, obj)` และ `apiResponse(res, result, null, httpStatus.CREATED)`
`node-express-esm-template/src/mappers/productline-mapper.js` → `map`	เรียก `toSummaryDTO(entity)` ไม่มี `this.` → ReferenceError; มี `import e from "express"` ที่ไม่ได้ใช้	`this.toSummaryDTO(entity)` และลบ import
`node-express-esm-template/src/middlewares/pagination-middleware.js`	`offset = (page - 1) * pageSize` ใช้ค่าก่อน clamp → ถ้า `limit=500` offset จะผิด; `parseInt('abc')` = `NaN`	ใช้ `limit` แทน `pageSize` และ fallback ถ้า NaN
`node-express-esm-template/src/repositories/brand-repo.js`	`ENTITY_NAME = "Office"` (copy มา)	เปลี่ยนเป็น `"Brand"`
`express-template/src/app.js`	มี error handler 2 ตัว ตัวที่ 2 (หลัง `listen`) ไม่มีวันถูกเรียก เพราะตัวแรกตอบไปแล้ว	เหลือตัวเดียว ไว้ท้ายสุด
`express-template/src/repositories/office-repo.js`	`new AppError(msg)` ไม่ส่ง statusCode แล้วค่อยตั้งทีหลัง	`new AppError(msg, 404, "NOT_FOUND")`
`basic-rest/package.json`	ไม่มี `"type": "module"` แต่ใช้ `import`	เพิ่ม `"type": "module"`
`test-express/package.json`	ไม่มี `"type": "module"`; script `nodemon app.js` ชี้	เพิ่ม `"type": "module"`, แก้เป็น `nodemon src/app.js`

ไฟล์	ปัญหา	วิธีแก้
	ไฟล์ผิด (ไฟล์จริงอยู่ `src/app.js`)	

> `int261-sample-exam/` เป็นโปรเจกต์ Spring Boot (Java) ของวิชา INT261 ไม่ใช่ INT161 — ใช้ได้แค่ดู schema ตาราง `brands` / `products` (sku เป็น UNIQUE → ใช้ทดสอบ 409)

---

12. Quick Reference

คำสั่งที่ใช้บ่อย

```
node file.js                # รันไฟล์
npm init -y                 # สร้าง package.json
npm install express mysql2 http-status
npm install -D nodemon
npm start                   # รันตาม scripts.start
curl -i -X METHOD url -H "Content-Type: application/json" -d '{...}'
```

Express Cheat Sheet

```
app.use(express.json())                 // parse JSON body
app.use('/prefix', router)              // mount router
router.get('/:id', handler)             // req.params.id
req.query.page                          // ?page=
req.body                                // JSON body
res.json(obj)                           // 200 + JSON
res.status(201).json(obj)               // status + JSON
res.status(204).end()                   // ไม่มี body
app.use((err, req, res, next) => {...}) // error handler (4 params, ท้ายสุด)
```

mysql2 Cheat Sheet

```
const [rows] = await db.query('SELECT * FROM t WHERE id = ?', [id])   // rows = array
const [result] = await db.query('INSERT INTO t SET ?', [obj])           // result.insertId
const [result] = await db.query('UPDATE t SET ? WHERE id = ?', [obj, id]) // result.affectedRows
const [[{ total }]] = await db.query('SELECT COUNT(*) as total FROM t')
const [rows] = await db.query('SELECT * FROM t LIMIT ? OFFSET ?', [limit, offset])
```

สูตร Pagination

```
offset      = (page - 1) * limit
totalPages  = Math.ceil(total / limit)
hasNextPage = page * limit < total
hasPrevPage = page > 1
```

คำถามทฤษฎีที่น่าออก (ทบทวน)

1. Node.js เป็นภาษาหรือไม่? → **ไม่ใช่ เป็น runtime**
2. Compiler vs Interpreter vs Hybrid ต่างกันอย่างไร / ตัวอย่างภาษา
3. CJS vs ESM — syntax, loading, tree-shaking, `__dirname` vs `import.meta.url`
4. 4 class หลักของ `node:http`
5. 6 REST constraints + 4 pillars ของ Uniform Interface
6. Stateless หมายความว่าอย่างไร
7. กฎตั้ง URI (noun, plural), collection / singleton / sub-collection
8. หน้าที่ Controller / Service / Repository (ควรมี/ห้ามมี)
9. ทำไมใช้ Pool และ Prepared Statement (กัน SQL Injection)
10. `pool.query()` คืนค่าอะไร (`[rows, fields]`)
11. `res.status()` ส่ง response เลยหรือไม่ → **ไม่ ต้อง chain .json/.send/.end**
12. Middleware คืออะไร, `next()` ทำอะไร, error middleware ต่างจากปกติอย่างไร (4 params) และต้องวางตรงไหน
13. Programming vs Operational errors, Error propagation
14. Status codes 200/201/204/400/401/403/404/409/412/500/503
15. สูตร offset, ทำไมต้อง limit สูงสุด, cursor pagination
16. DTO คืออะไร ทำไมใช้, Entity vs DTO (ห้ามส่ง password_hash)